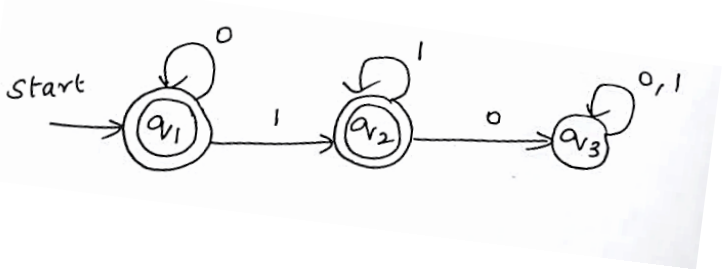
SAVEETHA SCHOOL OF ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

COMPUTER SCIENCE AND ENGINEERING

CSA13 – THEORY OF COMPUTATION

ASSIGNMENT – 2

		CO	BTL
1	Write closure properties of regular languages with respect to compliment, kleen star, reversal and substitution	2	4
2	Construct a Non Deterministic Finite automata with ε-transitions to represent the language denoted by the regular expression (a+b)*(ab)*	2	3
3	Give an informal description of the language denoted by the regular expression (0+1)*00	2	3
4	Justify that the language L consisting of all strings with an equal number of a's and b's over the alphabet **Σ**={a,b} is not regular using pumping lemma .	2	4
5	Apply Arden's Theorem and derive the regular expression for the language represented by the given automata 	2	3

Answer:

1.

Operation	Closure Property	Brief Mechanism / Construction
Complement ($\overline{L}$)	**Closed**	Swap accepting and non-accepting states in a complete DFA.
Kleene Star (L^*)	**Closed**	Add a new start state with an ϵ-transition to the old start state and loop back final states.
Reversal (L^R)	**Closed**	Reverse all transition arrows, swap start and final states in an NFA.
Substitution	**Closed**	Replace each terminal symbol with a regular language; results in a regular language.

Detailed Notes (Ready-to-Write Format)

1. Complement ($\overline{L}$)

- **Property: Closed**
- **Explanation:** If L is a regular language over an alphabet Σ, its complement $\overline{L} = \Sigma^* \setminus L$ is also regular.
- **Proof Idea:** Take a Deterministic Finite Automaton (DFA) that accepts L. Ensure the DFA is complete (all transition states are defined). To accept $\overline{L}$, simply invert the accept states: make all final states non-final, and all non-final states final. The resulting DFA accepts exactly $\overline{L}$.

2. Kleene Star (L^*)

- **Property: Closed**
- **Explanation:** If L is a regular language, then L^* (zero or more concatenations of strings from L) is also regular.
- **Proof Idea:** Start with an NFA that accepts L. Create a new start state S_{new} with an ϵ-transition to the original start state. Also, add ϵ-transitions from all original accepting states back to the original start state (to allow repetition) and make S_{new} an accepting state itself (to handle zero occurrences, i.e., the empty string ϵ).

3. Reversal (L^R)

- **Property: Closed**
- **Explanation:** The reversal of a regular language, $L^R = \{w_n \dots w_2 w_1 \mid w_1 w_2 \dots w_n \in L\}$, is regular.

- **Proof Idea:** Begin with an NFA for L.
    1. Reverse the direction of all transition arrows.
    2. Swap the roles: make the old start state the unique new accept state, and make the old accept state(s) the new start state (or add a new start state with ϵ-transitions to all old accept states).

4. Substitution

- **Property: Closed**
- **Explanation:** Let Σ and Γ be alphabets. A substitution s maps each symbol $a \in \Sigma$ to a regular language $s(a)$ over Γ. The substitution of a regular language L is $s(L) = \{w_1 w_2 \dots w_n \mid a_1 a_2 \dots a_n \in L \text{ and } w_i \in s(a_i)\}$. If L is regular and all languages $s(a)$ are regular, then $s(L)$ is regular.
- **Proof Idea:** Replace every transition labeled with symbol a in the automaton for L with the entire finite automaton that accepts the regular language $s(a)$. Since regular languages are closed under concatenation and union, substituting regular languages into a regular structure preserves regularity.

2.

Overview of Thompson's Construction Method

To construct an ϵ-NFA from a regular expression, we decompose the expression into basic building blocks using **Thompson's Construction**:

1. **Symbol:** $a \Longrightarrow$ (Start) $\xrightarrow{a}$ (Accept)
2. **Union ($R_1 + R_2$):** Introduce new start/final states with ϵ-branches to both components.
3. **Concatenation ($R_1 R_2$):** Connect the final state of R_1 to the start state of R_2 via an ϵ-transition.
4. **Kleene Star (R^*):** Add a start state branching to R and the final state via ϵ, with a loop-back ϵ-transition from the end of R to its start.

Here, $R = R_1 \cdot R_2$, where:

- $R_1 = (a + b)^*$
- $R_2 = (ab)^*$

2. Step-by-Step State Construction

Let the set of states in our final ϵ-NFA be $Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8, q_9, q_{10}, q_{11}\}$.

Part A: Constructing $(a + b)^*$

1. **Sub-automaton for a and b:**
    - For symbol a: $q_2 \xrightarrow{a} q_3$
    - For symbol b: $q_4 \xrightarrow{b} q_5$
2. **Union $(a + b)$:**
    - Create a start state q_1 and a final state q_6.

- Add ϵ-transitions: $q_1 \xrightarrow{\epsilon} q_2$, $q_1 \xrightarrow{\epsilon} q_4$, $q_3 \xrightarrow{\epsilon} q_6$, $q_5 \xrightarrow{\epsilon} q_6$.

3. **Kleene Star $(a + b)^*$:**
   - Create overall start state $q_0$and intermediate final state q_7.
   - Add ϵ-transitions:
     - $q_0 \xrightarrow{\epsilon} q_1$(Enter inner union)
     - $q_0 \xrightarrow{\epsilon} q_7$(Handle empty string ϵ)
     - $q_6 \xrightarrow{\epsilon} q_7$(Exit inner union)
     - $q_6 \xrightarrow{\epsilon} q_1$(Loop back for repetition)

Part B: Constructing $(ab)^*$

4. **Concatenation (ab):**
   - Symbol a: $q_9 \xrightarrow{a} q_{10}$
   - Symbol b: $q_{10} \xrightarrow{b} q_{11}$*(Combined aand bsequentially)*
5. **Kleene Star $(ab)^*$:**
   - Create start state $q_8$and overall accept state q_{11}.
   - Add ϵ-transitions:
     - $q_8 \xrightarrow{\epsilon} q_9$(Enter ab)
     - $q_8 \xrightarrow{\epsilon} q_{11}$(Handle empty string ϵ)
     - q_{11}loops back to $q_9$via an ϵ-transition for repetition of (ab).

Part C: Concatenating $(a + b)^*$and $(ab)^*$

6. **Joining Part A and Part B:**
   - Connect the exit of $(a + b)^*$(state q_7) to the entry of $(ab)^*$(state q_8) using an ϵ-transition:

$$q_7 \xrightarrow{\epsilon} q_8$$

$$q_7 \xrightarrow{\epsilon} q_8$$

3. Formal Definition of the ϵ-NFA

The resulting ϵ-NFA is defined as 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$:

- **States (Q):** $\{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8, q_9, q_{10}, q_{11}\}$
- **Alphabet (Σ):** $\{a, b\}$
- **Start State:** q_0

- **Accepting State (F):** $\{q_{11}\}$

4. State Transition Table

Present State	**Input a**	**Input b**	**Input ϵ**
$\rightarrow q_0$	$\emptyset$	$\emptyset$	$\{q_1, q_7\}$
q_1	$\emptyset$	$\emptyset$	$\{q_2, q_4\}$
q_2	$\{q_3\}$	$\emptyset$	$\emptyset$
q_3	$\emptyset$	$\emptyset$	$\{q_6\}$
q_4	$\emptyset$	$\{q_5\}$	$\emptyset$
q_5	$\emptyset$	$\emptyset$	$\{q_6\}$
q_6	$\emptyset$	$\emptyset$	$\{q_1, q_7\}$
q_7	$\emptyset$	$\emptyset$	$\{q_8\}$
q_8	$\emptyset$	$\emptyset$	$\{q_9, q_{11}\}$
q_9	$\{q_{10}\}$	$\emptyset$	$\emptyset$
q_{10}	$\emptyset$	$\{q_{11}\}$	$\emptyset$
$* q_{11}$	$\emptyset$	$\emptyset$	$\{q_9\}$

*Note: →indicates Start state; $*$$ indicates Accept state.**

3.

Informal Description

The regular expression *$(0 + 1)^ 00$** denotes the set of **all binary strings over the alphabet** $\{0,1\}$**that end with two consecutive zeros (**00**)**.

Component Breakdown:

1. $(0 + 1)^*$**:** This part matches any sequence of zeros and ones of any length (including the empty string ϵ). This acts as an arbitrary prefix.
2. 00**:** This part ensures that the string strictly terminates with the exact suffix 00.

Examples

- **Strings belonging to the language (L):**
    - 00 (from ϵfollowed by 00)
    - 000 (from 0 followed by 00)
    - 100 (from 1 followed by 00)
    - 10100 (from 101 followed by 00)

- 01100 (from 011 followed by 00)

- **Strings NOT belonging to the language:**
  - 0 (does not end in two zeros)
  - 1 (does not end in two zeros)
  - 01 (ends in 01, not 00)
  - 001 (ends in 01, not 00)

4.

To prove that the language $L = \{w \in \{a, b\}^* \mid N_a(w) = N_b(w)\}$(where $N_a(w)$and $N_b(w)$denote the number of a's and b's in w) is not regular, we use Proof by Contradiction via the **Pumping Lemma for Regular Languages**.

Step 1: Statement of the Pumping Lemma

If $L$is a regular language, then there exists a constant $p \geq 1$(the pumping length) such that any string $s \in L$with $|s| \geq p$can be split into three parts, $s = xyz$, satisfying the following three conditions:

1. $|xy| \leq p$
2. $|y| > 0$(i.e., $y \neq \epsilon$)
3. $xy^i z \in L$for all $i \geq 0$

Step 2: Assumption for Contradiction

Assume for the sake of contradiction that the language $L$is regular. Therefore, Lmust satisfy the Pumping Lemma with some pumping length p.

Step 3: Choosing a Suitable String s

Select the string:

$$s = a^p b^p$$

$$s = a^p b^p$$

Now let's check the criteria for string s:

- $s$consists of pa's followed by pb's.
- The number of a's equals the number of b's, so $s \in L$.
- The length of $s$is $|s| = 2p \geq p$, satisfying the conditions of the Pumping Lemma.

Step 4: Applying the Pumping Lemma Constraints

According to the Pumping Lemma, $s$can be partitioned into $s = xyz$such that:

1. $|xy| \leq p$
2. $|y| > 0$

Since the first $p$characters of $s$consist entirely of the letter a, condition 1 ($|xy| \leq p$) forces both $x$and $y$to consist **only of a's**.

Therefore, we can express the substrings as:

- $x = a^k$
- $y = a^m$(where $m > 0$because $|y| > 0$)
- $z = a^{p-k-m}b^p$

This means $y$consists of $m$copies of the symbol $a(m \geq 1)$.

Step 5: Pumping $y$to Create a Contradiction

According to condition 3 of the Pumping Lemma, xy^izmust belong to $L$for all integers $i \geq 0$.

Let us choose $i = 2$(pumping up):

$$xy^2z = xyyz$$

$$xy^2z = xyyz$$

$$xy^2z = a^k \cdot (a^m)^2 \cdot a^{p-k-m}b^p = a^{p+m}b^p$$

$$xy^2z = a^k \cdot (a^m)^2 \cdot a^{p-k-m}b^p = a^{p+m}b^p$$

Now, count the number of a's and b's in xy^2z:

- Number of a's = $p + m$
- Number of b's = p

Since $m > 0$, we have $p + m \neq p$. The number of a's is strictly greater than the number of b's in xy^2z.

Thus, $xy^2z \notin L$, which violates Condition 3 of the Pumping Lemma.

Conclusion

Our initial assumption that $L$is regular led to a contradiction. Therefore, by the Pumping Lemma, the language Lconsisting of all strings with an equal number of a's and b's is **not regular**.

5.

1. Analysis of the Finite Automaton

Let the states of the given automaton be q_1, q_2,and q_3:

- **Start State:** q_1
- **Final (Accepting) States:** $q_1$and q_2(indicated by double circles)
- **Dead / Trap State:** q_3(from which no final state can be reached, as it only loops to itself on 0,1)

State Transitions from the Diagram:

- **From state q_1:**
    - Loops on 0 →goes to q_1

- Transitions on 1 →goes to q_2
- Reaches start (implicitly ϵ) → ϵ

- **From state q_2:**
  - Transitions from $q_1$on 1 → $q_1 1$
  - Loops on 1 →goes to q_2
- **From state q_3:**
  - Transitions from $q_2$on 0 → $q_2 0$
  - Loops on 0, 1 →goes to itself

2. Setting Up State Equations

Using the transitions, we write the equation for each state representing the set of strings accepted starting from that state:

1. **For state q_1:**

$$q_1 = q_1 0 + \epsilon$$

$$q_1 = q_1 0 + \epsilon$$

2. **For state q_2:**

$$q_2 = q_1 1 + q_2 1$$

$$q_2 = q_1 1 + q_2 1$$

3. **For state q_3(Trap state - can be ignored for final acceptance since it has no final states):**

$$q_3 = q_2 0 + q_3 (0 + 1)$$

$$q_3 = q_2 0 + q_3 (0 + 1)$$

3. Applying Arden's Theorem

Arden's Theorem State: If $R = Q + RP$where $\epsilon \notin P$, then the unique solution is $R = QP^*$.

Step A: Solve for q_1

Equation:

$$q_1 = q_1 0 + \epsilon$$

$$q_1 = q_1 0 + \epsilon$$

Applying Arden's Theorem (where $R = q_1$, $Q = \epsilon$, $P = 0$):

$$q_1 = \epsilon 0^* = 0^*$$

$$q_1 = \epsilon 0^* = 0^*$$

Step B: Substitute q_1 into q_2 and Solve for q_2

Equation for q_2:

$$q_2 = q_2 1 + q_1 1$$

$$q_2 = q_2 1 + q_1 1$$

Substitute $q_1 = 0^*$:

$$q_2 = q_2 1 + 0^* 1$$

$$q_2 = q_2 1 + 0^* 1$$

Applying Arden's Theorem (where $R = q_2$, $Q = 0^*1$, $P = 1$):

$$q_2 = (0^*1)1^*$$

$$q_2 = (0^*1)1^*$$

$$q_2 = 0^*1^+ \text{ (or } 0^*11^*)$$

$$q_2 = 0^*1^+ \quad \text{(or } 0^*11^*)$$

4. Final Regular Expression

Since both q_1 and q_2 are final (accepting) states, the total language accepted by the automaton is the union of the expressions for q_1 and q_2:

$$\text{Total Language } L = q_1 + q_2$$

$$\text{Total Language } L = q_1 + q_2$$

$$L = 0^* + 0^*11^*$$

$$L = 0^* + 0^*11^*$$

We can factor out 0^*:

$$L = 0^*(\epsilon + 11^*)$$

$$L = 0^*(\epsilon + 11^*)$$

Since $\epsilon + 11^* = 1^*$, we get:

$$L = 0^*1^*$$

$$L = 0^*1^*$$

Final Answer:

The regular expression for the given automaton is *$0^ 1^*$* (which represents any number of 0s followed by any number of 1s).